

NPTL and the Linux Kernel

A comprehensive, undergraduate-friendly guide to how Linux implements POSIX threads: from `pthread_create()` in your C program, through the glibc thread library, down to `clone()`, `futex()` and the kernel scheduler.

Generated for CMSC 125 ICS UPLB

Reviewed by JACH

Includes diagrams, annotated kernel and glibc walk-throughs, four hands-on labs with real output, and review questions with answers.

Contents

Before You Begin	4
The whole story in one paragraph	4
Prerequisites	4
Roadmap	4
Why Threads?	6
1.1 A quick recap: processes	6
1.2 What is a thread?	6
1.3 Why use threads?	6
1.4 The price: shared memory means shared bugs	7
Threading Models: Who Schedules What?	8
POSIX Threads, NPTL, and a Bit of History	10
3.1 Two names, two ideas	10
3.2 Who does what?	10
3.3 History: how Linux got NPTL	11
Threads in the Kernel: Everything Is a Task	13
4.1 The surprising truth	13
4.2 <code>clone()</code> : one system call to create them all	13
4.3 PID, TGID, and TID: solving the naming puzzle	14
4.4 Inside <code>clone()</code> : a simplified kernel walk-through	14
4.5 Thread exit and thread groups	15
4.6 Seeing threads from the shell	15
Kernel Features That Made NPTL Possible	16
Anatomy of <code>pthread_create()</code>	17
6.1 Step 1 and 2: the thread's stack	17
6.2 Step 3: the thread descriptor (<code>struct pthread</code>)	18
6.3 Step 4 to 6: calling into the kernel	18
6.4 Step 7: the new thread starts	19
6.5 Joining, detaching, and the stack cache	19
Futexes: Fast Locks With Kernel Help	20
7.1 The problem	20
7.2 What a futex is	20

7.3 A mutex built on a futex	21
7.4 Everything else builds on futexes	22
Signals, Cancellation, fork(), and exec()	23
8.1 Signals	23
8.2 Thread cancellation	23
8.3 Internal signals used by NPTL	23
8.4 fork() in a multithreaded program	23
8.5 exec() and exit	24
Thread-Local Storage (TLS)	25
9.1 Using TLS	25
9.2 How the hardware and kernel help	25
Scheduling and System Limits	26
10.1 The kernel schedules threads	26
10.2 Controls you can use	26
10.3 Why does <code>pthread_create</code> fail? (EAGAIN)	26
Hands-On Labs	28
Lab 1: Same PID, different TIDs	28
Lab 2: Make a race condition, then fix it with a mutex	28
Lab 3: Watch clone and futex with strace	29
Lab 4: Look at threads from outside	30
Modern Developments and Alternatives	32
12.1 What changed since 2003?	32
12.2 Other C libraries	32
12.3 Alternatives to "one thread per task"	32
Review, Misconceptions, and Reference	33
13.1 Common misconceptions	33
13.2 Where to find it in the source	33
13.3 Review questions	34
13.4 Answer key	34
13.5 Glossary	35
13.6 References and further reading	35

START HERE

Before You Begin

Every time you open a browser tab, run a web server, or play a game that keeps its graphics, audio and network code running at the same time, threads are doing the work. On Linux, the software that makes threads possible is called **NPTL**, the **Native POSIX Thread Library**. It is not a separate program you install. It is a part of the C library (glibc) that every ordinary Linux program already uses.

This guide explains what NPTL is, why it was created, and, most importantly for an operating systems course, **how it cooperates with the Linux kernel**. We will keep asking one question: *which part of the work happens in user space (the library), and which part happens in the kernel?*

The whole story in one paragraph

Linux does not have a special "thread" object. The kernel only knows about **tasks** (`task_struct`). A **thread** is simply a task that was created with the `clone()` system call using flags that say *share my memory, my open files, and my signal handlers with me*. NPTL is the library that calls `clone()` with the right flags, allocates a stack for each thread, sets up per-thread storage, and provides the friendly `pthread_*` functions. When threads need to wait for each other (mutexes, condition variables), NPTL uses another kernel feature called the **futex**, which lets threads go to sleep in the kernel only when there is real contention. Because each thread is a kernel task, the ordinary Linux scheduler runs threads on all your CPU cores. This design is called the **1:1 model**.

Prerequisites

- Basic C: pointers, structs, function pointers (thread functions take a `void *`).
- OS basics from lecture: process, address space, context switch, system call, user vs kernel mode.
- Comfort with a Linux shell. The labs use `gcc`, `ps`, and `strace`.

Roadmap

Chapter	Question it answers
1 Why threads?	What is a thread and what does it share with its process?
2 Threading models	How can an OS map user threads onto CPUs? Why did Linux choose 1:1?
3 POSIX, NPTL, history	What is the difference between pthreads and NPTL? What came before?
4 Threads in the kernel	What does <code>clone()</code> do? What are PID, TGID and TID?
5 Kernel support	Which kernel features had to exist for NPTL to be possible?
6 Anatomy of <code>pthread_create</code>	Step by step, what happens when you create a thread?
7 Futexes	How can a mutex be fast when uncontended and still let threads sleep?
8 Signals, cancel, fork, exec	How do process-level operations behave with many threads?
9 Thread-local storage	How does each thread get its own <code>errno</code> and <code>__thread</code> variables?
10 Scheduling and limits	How are threads scheduled? Why does <code>pthread_create</code> fail?
11 Hands-on labs	Observe all of the above on a real machine.

Chapter	Question it answers
12 Modern developments	What has changed since 2003? What are the alternatives?
13 Review	Misconceptions, exam-style questions, answers, glossary, references.

NOTE — ABOUT THE EXAMPLES

Sample outputs in this guide were produced on Ubuntu 24.04 with glibc 2.39 on a Linux 6.x kernel. Your numbers (addresses, TIDs, timings) will differ, but the *patterns* will match. Kernel and glibc internals shown here are **simplified** for learning; real source files contain many extra checks and architecture-specific paths.

CHAPTER 1

Why Threads?

1.1 A quick recap: processes

A **process** is a running program. The OS gives each process its own **virtual address space** (code, data, heap, stack), a table of open files, signal handlers, and an identity (a process ID). Processes are isolated from each other: one process cannot read another's memory by accident. That isolation is great for safety but expensive for cooperation. Sharing data between processes needs pipes, sockets, or shared-memory setup, and creating a process with `fork()` means copying (or copy-on-write mapping) an entire address space.

1.2 What is a thread?

A **thread** is an independent flow of execution *inside* a process. All threads of a process live in the same address space, so they can read and write the same global variables and heap objects directly. Each thread only needs its own small amount of private state to keep track of *where it is* in the code.

KEY IDEA

Analogy: a process is a **house**. Threads are the **housemates**. They share the kitchen, the living room and the Wi-Fi (memory, open files, signal handlers). But each housemate has a private bedroom (their own stack), and their own to-do list and current step (their own registers and program counter).

Shared by all threads in a process	Private to each thread
Code (text) and global/static data	Stack (local variables, return addresses)
Heap (mal located memory)	CPU register values, including the program counter
Open file descriptors	Thread ID (TID)
Signal handlers (dispositions)	Signal mask (which signals are blocked)
Current working directory, umask	Thread-local storage (TLS), including errno
Process ID and address-space mappings	Scheduling state, priority, CPU affinity

1.3 Why use threads?

- **Responsiveness.** A GUI keeps handling clicks while another thread loads a file.
- **Parallelism.** On a multicore CPU, threads of one program can truly run at the same time. Splitting a big computation across 8 cores can make it up to 8x faster.
- **Cheap sharing.** Threads share memory, so passing data is just reading a variable.
- **Lower creation cost.** Creating a thread avoids duplicating an address space, so it is typically much cheaper than creating a process.
- **Simpler blocking I/O.** A server can dedicate one thread per connection and use simple blocking `read()/write()` calls; while one thread waits, others keep running.

1.4 The price: shared memory means shared bugs

Because threads share memory, two threads can touch the same variable at the same time. Consider `counter++`. It looks like one step, but the CPU does three:

Thread A	Thread B	counter in memory
-----	-----	-----
load counter (reads 5)		5
	load counter (reads 5)	5
add 1 (now 6)		5
	add 1 (now 6)	5
store counter (writes 6)		6
	store counter (writes 6)	6 <-- should be 7!

This is a **race condition** (a lost update). The cure is **synchronization**: mutexes, condition variables, semaphores, atomics. Implementing these efficiently is a big part of what NPTL and the kernel's `futex` support are for (Chapter 7).

CHAPTER 2

Threading Models: Who Schedules What?

There are two "levels" where a thread can exist. **User-level threads** are created and switched by a library inside the process; the kernel does not know about them. **Kernel-level threads** are known to the kernel and scheduled by it onto CPUs. An OS must decide how user threads map to kernel threads. There are three classic answers.

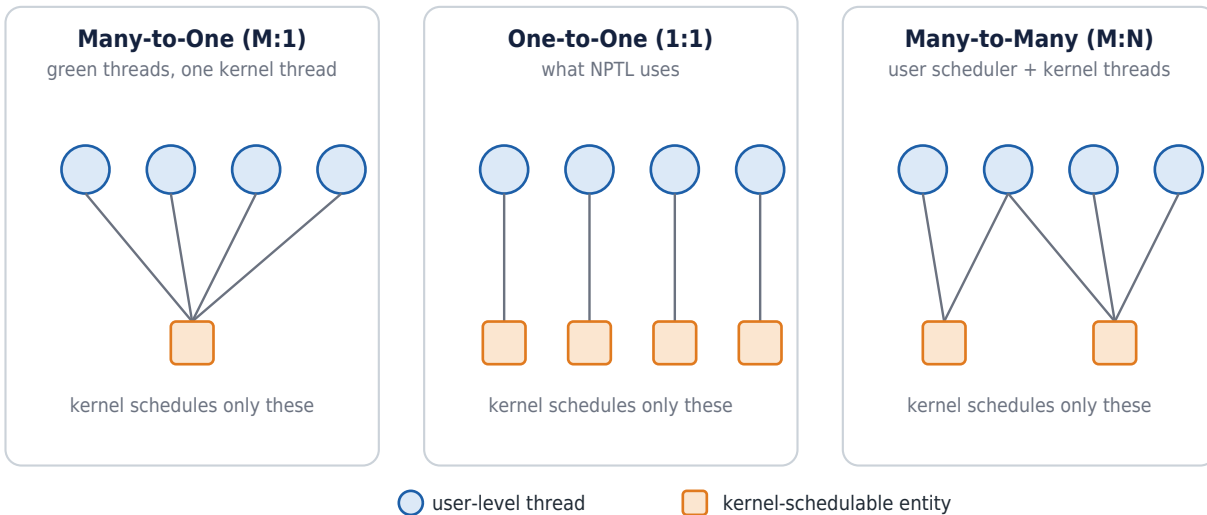


Figure 2.1: The three classic mappings between user threads and kernel-schedulable entities.

Model	How it works	Advantages	Disadvantages
M:1 (many-to-one)	All user threads share one kernel thread. A library switches between them.	Very cheap thread switch (no system call). Works even on kernels with no thread support.	No parallelism: only one CPU used. One blocking system call blocks every thread.
1:1 (one-to-one)	Each user thread is its own kernel thread (task). NPTL uses this.	True parallelism on many cores. Blocking one thread does not block others. Kernel scheduler does all the work; the library is simple.	Creation and switching involve the kernel (a bit more costly). Very large thread counts use kernel memory.
M:N (many-to-many)	M user threads are multiplexed on N kernel threads by a two-level scheduler.	Cheap threads <i>and</i> parallelism, in theory.	Very complex: two schedulers must cooperate, and they fight each other. Hard to get signals, priorities, and blocking calls right.

Some systems (early Solaris, and the IBM **NGPT** project for Linux) tried M:N and later moved away from it because of the complexity. Linux chose **1:1**, mainly because Linux tasks are already very cheap to create and switch, so the extra complexity of M:N was not worth it.

KEY IDEA

In the 1:1 model there is **no thread scheduler in the library**. The Linux kernel scheduler treats every thread exactly like any other runnable task. That is why threads automatically spread across your CPU cores.

NOTE — NOT TO BE CONFUSED

"Green threads" or "goroutines" (Go) are user-level constructs built *on top of* kernel threads, so they are M:N. That is a design choice of the language runtime. NPTL itself does not do M:N.

CHAPTER 3

POSIX Threads, NPTL, and a Bit of History

3.1 Two names, two ideas

POSIX Threads (pthreads) is an *API specification* (IEEE POSIX.1c, 1995) that says what functions exist and how they must behave: `pthread_create`, `pthread_join`, `pthread_mutex_lock`, `pthread_cond_wait`, and so on. It says nothing about *how* to implement them.

NPTL is an *implementation* of that API for Linux. It lives inside **glibc** (the GNU C library) and relies on kernel features (Chapter 5). When you compile with `gcc -pthread` and call `pthread_create()`, you are calling NPTL code.

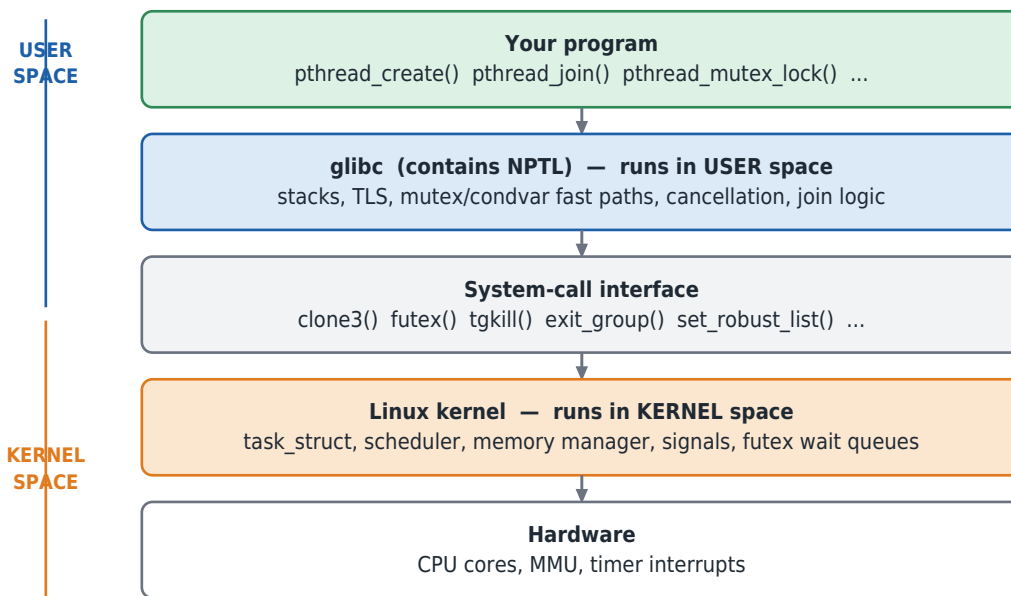


Figure 3.1: Where NPTL sits. The library is user-space code that talks to the kernel through a few system calls.

3.2 Who does what?

NPTL / glibc (user space)	Linux kernel
Provides the <code>pthread_*</code> API	Creates tasks (<code>clone</code>), tracks them in <code>task_struct</code>
Allocates thread stacks (<code>mmap</code>) and guard pages	Schedules tasks on CPUs; handles context switches
Sets up each thread's TLS and descriptor (<code>struct pthread</code>)	Shares (or copies) memory, files and signal handlers according to <code>clone</code> flags
Implements mutex, condvar, rwlock, semaphore fast paths with atomic instructions	Puts threads to sleep / wakes them via <code>futex()</code> wait queues
Implements join, detach, cancellation, thread-specific data	Delivers signals to the correct task; tears down tasks on exit

NPTL / glibc (user space)	Linux kernel
Caches finished thread stacks for reuse	Enforces limits: <code>RLIMIT_NPROC</code> , <code>threads-max</code> , <code>pid_max</code> , <code>cgroups</code>

3.3 History: how Linux got NPTL

Early Linux (before the 2.6 kernel) had no real thread support. The first widely used library was **LinuxThreads** (Xavier Leroy, 1996). It was a clever hack: it used `clone()` to make one kernel task per thread, but the kernel of that era had no notion of a thread group, so the library had to fake many things. As programs grew more demanding (big servers, Java virtual machines), the cracks showed.

Problem in LinuxThreads	Consequence	How NPTL fixes it
Every thread had its own PID; there was no thread-group concept	<code>ps</code> and <code>top</code> showed each thread as a separate process; <code>getpid()</code> returned different values in one program	Kernel thread groups: all threads share one TGID, which <code>getpid()</code> returns
Signals were emulated with internal signals (<code>SIGUSR1/SIGUSR2</code>) and a manager thread	Signals to a process could be delivered wrongly; extra manager thread cost; not POSIX-compliant	Kernel handles process-wide vs thread-directed signals; no manager thread
Synchronization used signals to wake sleeping threads	Slow, and wake-ups could be lost or delayed	Futexes: efficient kernel-assisted sleeping and waking
Thread count was limited to roughly a thousand or a few thousand	Big servers hit the ceiling quickly	Kernel and library scale to tens of thousands or more (limited by memory and system limits)
Poor scalability of thread creation and exit	Creating many threads was very slow	New kernel scheduler ($O(1)$), streamlined <code>clone</code> , stack caching

Two projects competed to replace LinuxThreads: **NGPT** (IBM, an M:N design) and **NPTL** (Ulrich Drepper and Ingo Molnar at Red Hat, a 1:1 design built together with kernel changes). NPTL won: it was simpler, faster in benchmarks, and matched the improved kernel. The design paper reported creating 100,000 threads in about two seconds on its test machine, versus roughly 15 minutes with LinuxThreads. NPTL shipped in Red Hat Linux 9 (2003) and became the standard with Linux 2.6 and glibc 2.3.x.

Timeline

Year	Event
1995	POSIX.1c standardizes the pthreads API
1996	LinuxThreads appears (Xavier Leroy)
2002	Futex support is developed in the Linux 2.5 series; NPTL design starts alongside the $O(1)$ scheduler and thread-group work
2003	NPTL ships (Red Hat Linux 9, glibc 2.3.2); Linux 2.6.0 released in December
2006	Robust futexes (Linux 2.6.17) and priority-inheritance futexes (2.6.18)
2007	CFS replaces the $O(1)$ scheduler (Linux 2.6.23)
2019	<code>clone3()</code> system call (Linux 5.3)

Year	Event
2021	glibc 2.34 merges <code>libpthread</code> into <code>libc</code>
2022	<code>futex_waitv()</code> (Linux 5.16)
2023	EEVDF replaces CFS as the default fair scheduler (Linux 6.6)

CHAPTER 4

Threads in the Kernel: Everything Is a Task

4.1 The surprising truth

If you read the Linux kernel source looking for a "thread" structure, you will not find one. Linux represents **both processes and threads with the same structure: `struct task_struct`**. The kernel's own documentation and code often just say "task". A traditional single-threaded process is a task. A multithreaded process is *several tasks that share resources*.

KEY IDEA

Process vs thread on Linux is only a question of what is shared. Two tasks that share the same address space, file table and signal handlers, and belong to the same thread group, are what the rest of the world calls threads of one process.

4.2 `clone()`: one system call to create them all

Classic `fork()` copies almost everything. Linux provides a more general call, `clone()`, in which the caller chooses **what to share with the new task** using flag bits. `fork()` is essentially `clone()` with no sharing flags, `vfork()` shares the address space until `exec`, and `pthread_create()` uses `clone()` with lots of sharing:

Flag	Meaning when set	Why NPTL sets it
<code>CLONE_VM</code>	Share the same address space (same <code>mm_struct</code>)	Threads see the same memory
<code>CLONE_FS</code>	Share cwd, root directory, umask	Same filesystem view
<code>CLONE_FILES</code>	Share the open-file table	A file opened by one thread is usable by all
<code>CLONE_SIGHAND</code>	Share the table of signal handlers	One <code>signal()</code> or <code>sigaction()</code> applies to all threads
<code>CLONE_THREAD</code>	Put the new task in the caller's thread group (same TGID)	Makes the group look like one process
<code>CLONE_SYSVSEM</code>	Share System V semaphore undo values	POSIX requires it
<code>CLONE_SETTLS</code>	Set the new task's thread-local-storage pointer	Each thread gets its own TLS/TCB
<code>CLONE_PARENT_SETTID</code>	Write the new TID into a given user address	Parent learns the TID immediately
<code>CLONE_CHILD_CLEARTID</code>	When the task exits, zero a given address and futex-wake it	This is how <code>pthread_join</code> gets woken (Chapter 6)

NOTE

Kernel rules: `CLONE_THREAD` requires `CLONE_SIGHAND`, and `CLONE_SIGHAND` requires `CLONE_VM`. You cannot build a thread group that does not share memory.

4.3 PID, TGID, and TID: solving the naming puzzle

The kernel gives every *task* a unique number. The naming is confusing because the kernel's internal names differ from what user programs see:

What you call it	Kernel field	Returned by	Meaning
Process ID (PID)	<code>tgid</code>	<code>getpid()</code>	Same for all threads of a process. Equals the TID of the first thread.
Thread ID (TID)	<code>pid</code>	<code>gettid()</code> (via <code>syscall</code>)	Unique per task, i.e. per thread.
pthread ID	(user-space only)	<code>pthread_self()</code>	A library-level handle. On glibc it is the address of the thread's descriptor, not the kernel TID.

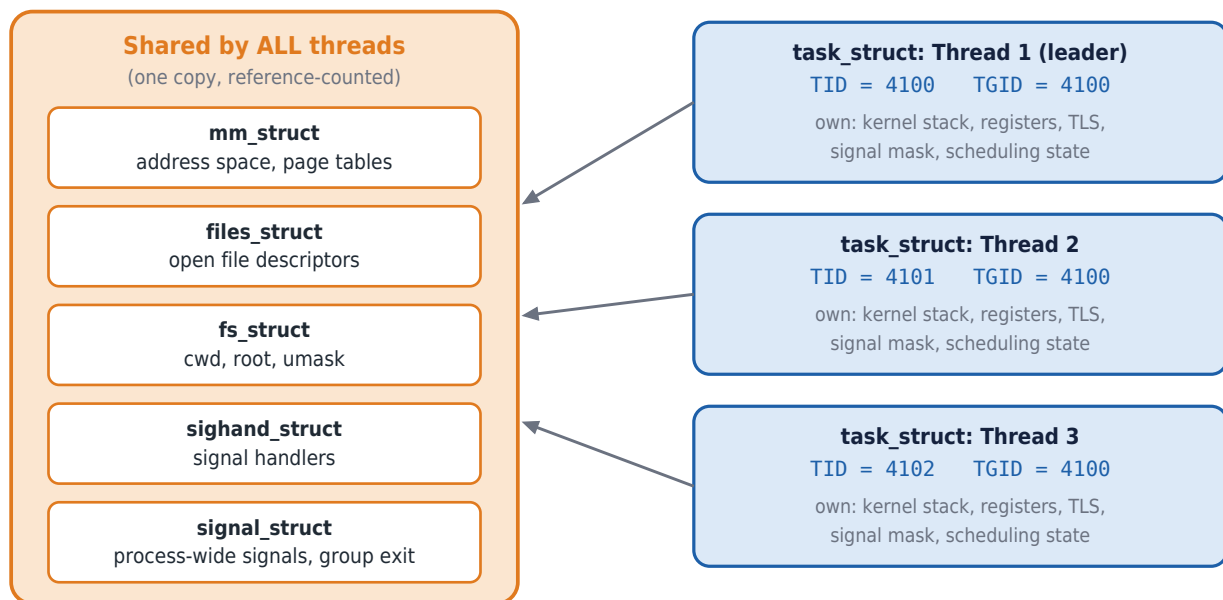


Figure 4.1: Three threads of one process. Each is its own `task_struct` with its own TID, but they all point at the same shared kernel structures and have the same TGID.

4.4 Inside `clone()`: a simplified kernel walk-through

The `clone/clone3` system calls end up in `kernel_clone()` in `kernel/fork.c`, which calls `copy_process()`. The heart of it, drastically simplified:

```

clone3(args)                                // syscall entry
-> kernel_clone()
    -> copy_process(clone_flags, ...)
        p = dup_task_struct(current);        // new task_struct + new kernel stack
        copy_files(flags, p);                // CLONE_FILES ? refcount++ : duplicate table
        copy_fs(flags, p);                   // CLONE_FS ? refcount++ : duplicate
        copy_sighand(flags, p);              // CLONE_SIGHAND ? refcount++ : duplicate
        copy_signal(flags, p);
        copy_mm(flags, p);                   // CLONE_VM ? share mm_struct : COPY page tables
        p->pid = alloc_pid();                 // a brand-new TID
        if (flags & CLONE_THREAD) {
            p->tgid = current->tgid;           // same "process ID" as the caller
            p->group_leader = current->group_leader;
            /* link p into the thread group's list */
        }
        copy_thread(p, ...);                  // registers, user stack pointer, TLS (CLONE_SETTLS)
    -> wake_up_new_task(p);                   // make it runnable: scheduler picks a CPU

```

Notice the pattern: for each resource, the flag chooses between **sharing** (increment a reference count; cost is tiny) and **copying** (much more work). A thread is cheap precisely because almost everything is shared.

4.5 Thread exit and thread groups

- `exit()` on a thread (system call `exit`) ends **only that task**.
- `exit_group()` ends **every task in the thread group**. The C library's `exit()` and returning from `main()` both lead here, which is why the whole process dies when `main` returns even if other threads are still running.
- If the *leader* thread (the one with TID = TGID) finishes first, it stays as a zombie until the whole group is done, because its ID is the process's ID.
- A fatal signal such as `SIGSEGV` (unhandled) or `SIGKILL` terminates the **entire thread group**, not just one thread.

4.6 Seeing threads from the shell

Threads appear in `/proc` under `/proc/<PID>/task/<TID>`. `ps -T -p <PID>` lists the threads of a process (the `SPID` column is the TID), and `top -H` shows threads instead of processes. Chapter 11 has a lab with real output.

CHAPTER 5

Kernel Features That Made NPTL Possible

NPTL is often described as a *library*, but the truth is that it was a **joint project between the library and the kernel**. During the Linux 2.5 development series, many kernel features were added specifically so a fast, POSIX-conforming 1:1 library could exist.

Kernel feature	What it does	Why NPTL needs it
Thread groups (<code>CLONE_THREAD</code> , TGID)	Groups tasks so they share one process identity.	<code>getpid()</code> , <code>kill()</code> , <code>exit_group()</code> and <code>wait()</code> behave as POSIX requires. No manager thread needed.
<code>futex()</code>	Fast user-space mutex support: the kernel only sleeps or wakes a thread on request.	Efficient mutexes, condvars, semaphores, barriers, and <code>pthread_join</code> .
<code>exit_group()</code>	Terminate all threads in the group at once.	Process <code>exit()</code> while other threads are running.
<code>tgkill()</code> (and <code>tkill</code>)	Send a signal to a specific thread in a specific thread group.	<code>pthread_kill()</code> , <code>pthread_cancel()</code> .
Thread-aware signal delivery	Process-directed signals go to <i>some</i> thread that does not block them; per-thread masks and pending sets.	POSIX signal semantics.
TLS support (<code>CLONE_SETTLS</code> , <code>set_thread_area</code> , <code>arch_prctl</code>)	Give each task its own thread-pointer register value (FS/GS base on x86).	Fast <code>__thread</code> variables and <code>errno</code> .
<code>set_tid_address()</code> / <code>CLONE_CHILD_CLEARTID</code>	Kernel clears a user-memory word and wakes futex waiters when the task exits.	<code>pthread_join</code> without polling or signals.
O(1) scheduler (later CFS, EEVDF)	Scheduling cost independent of the number of runnable tasks.	Thousands of threads without slowing down the scheduler.
Scalable PID allocation, cheaper <code>clone</code>	Faster creation and destruction of tasks.	Fast thread creation (the design paper's headline result).
Robust futexes (2.6.17)	If a thread dies holding a mutex, the kernel marks it so waiters can recover (<code>EOWNERDEAD</code>).	<code>PTHREAD_MUTEX_ROBUST</code> .
Priority-inheritance futexes (2.6.18)	The kernel boosts the priority of a lock holder blocking a higher-priority waiter.	<code>PTHREAD_PRIO_INHERIT</code> (real-time use).
<code>clone3()</code> (5.3)	Extensible replacement for <code>clone()</code> with a struct of arguments.	Used by glibc since 2.34 (you will see it in <code>strace</code>).

KEY IDEA

The recipe for a 1:1 threading library: (1) tasks that can share everything, (2) a way to sleep and wake without burning CPU or the kernel on every lock, (3) correct signal and exit semantics, (4) per-thread storage. The kernel supplies the mechanisms; NPTL supplies the policy and the `pthread_*` API.

CHAPTER 6

Anatomy of pthread_create()

Let us follow one call, `pthread_create(&t, NULL, worker, arg)`, from beginning to end.



Figure 6.1: The life of a `pthread_create()` call. Steps 1 to 4 are in glibc, 5 and 6 are the kernel, 7 is the new thread.

6.1 Step 1 and 2: the thread's stack

Every thread needs its own stack, since it runs its own chain of function calls. NPTL asks the kernel for memory with `mmap()`. The default size comes from the shell's stack limit (`ulimit -s`, usually **8 MB**). It is *virtual* memory: physical pages are only allocated when the thread actually touches them, so 8 MB does not mean 8 MB of RAM.

At the lowest addresses of the mapping, NPTL places a **guard page** with no permissions (`PROT_NONE`). Since stacks grow downward, a runaway recursion hits the guard page and causes an immediate `SIGSEGV`, instead of silently overwriting a neighbouring thread's memory.

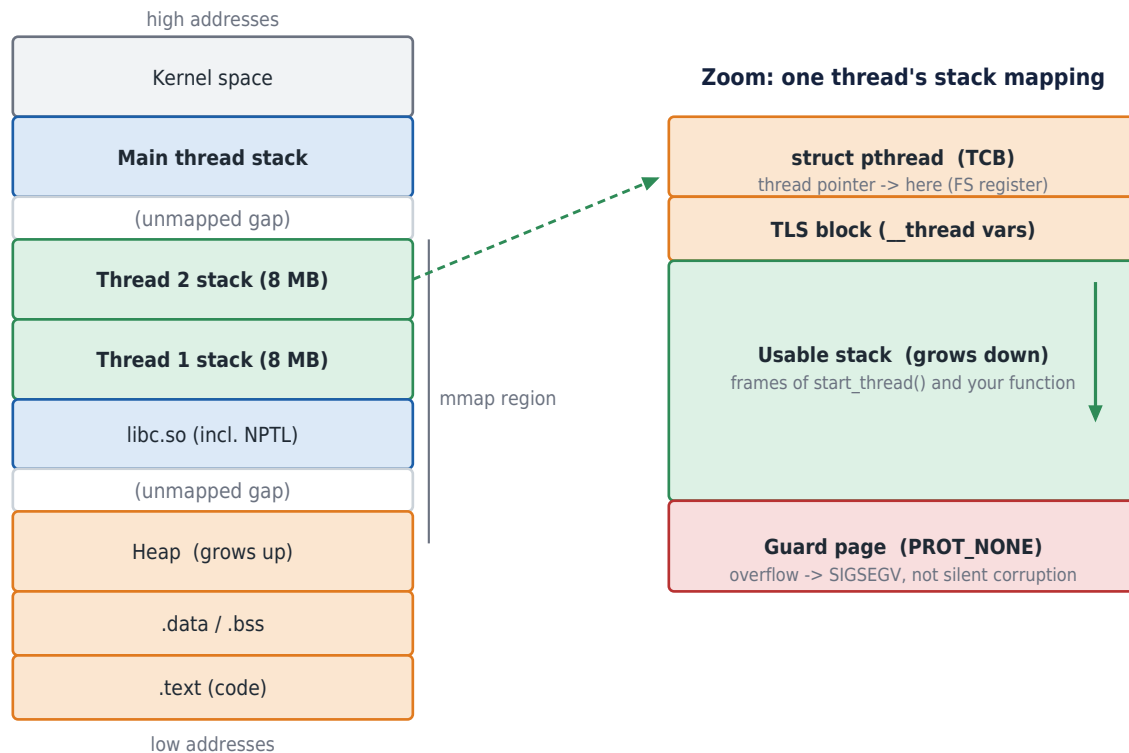


Figure 6.2: Address space of a multithreaded process (left) and one thread's stack region (right). The thread descriptor and TLS live at the top of the thread's own stack mapping.

6.2 Step 3: the thread descriptor (`struct pthread`)

glibc keeps all bookkeeping for a thread in a structure called `struct pthread`, also called the **TCB** (thread control block). It is placed at the top of the new thread's stack mapping and stores: the kernel TID, a pointer to itself, the start routine and its argument, the cancellation state, the return value (for `join`), the detach state, and a list of locks the thread holds (for robust mutexes). The TLS block sits right next to it (Chapter 9).

6.3 Step 4 to 6: calling into the kernel

glibc now invokes `clone3()` with the sharing flags from Table 4.2, the new stack, and the TLS pointer. The kernel runs `copy_process()` (Section 4.4), sets the new task's thread pointer, writes the new TID into the descriptor (`CLONE_PARENT_SETTID`), and hands the task to the scheduler. The parent returns from `clone3` with the new TID and `pthread_create` returns 0.

You can watch this happen. Running the program from Lab 1 under `strace -f` shows (trimmed for readability):

```
mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) = 0x7f8e369ff000
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|
      CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARID,
      exit_signal=0, stack=0x7f8e369ff000, stack_size=0x7fff80, tls=0x7f8e371ff6c0}
      => {parent_tid=[197]}, 88) = 197
```

Read it like a detective. `8392704` bytes is 8 MiB plus one 4 KiB page (the guard page). The flags are exactly the ones from Section 4.2, and note `exit_signal=0`: unlike a child process, a thread does not send `SIGCHLD` to anyone when it ends. The `tls` value is an address near the top of the new stack mapping (the TCB). The

return value 197 is the new thread's TID.

6.4 Step 7: the new thread starts

The new task begins in a small glibc function (`start_thread`) that is running on the new stack. It registers its robust-futex list with the kernel (`set_robust_list`, visible in `strace`), then calls your `worker(arg)`. When `worker` returns, `start_thread` stores the return value in the descriptor, runs thread-specific-data destructors, and exits the thread with the `exit` system call (not `exit_group`).

6.5 Joining, detaching, and the stack cache

`pthread_join(t)` must block until thread `t` finishes. It does this with `CLONE_CHILD_CLEARTID`: the kernel has been told the address of the descriptor's TID field. `pthread_join` calls `futex(WAIT)` on that address, sleeping as long as the value still equals the target's TID. When the target exits, the kernel itself writes 0 to that address and does a futex wake, which unblocks the joiner. No signal and no polling are needed. You can see this in Lab 3's trace:

```
189 futex(0x7f25b01ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 190, NULL, ...) <unfinished ...>
190 exit(0)                                = ?          <-- thread 190 ends; kernel clears the TID
189 <... futex resumed>                    = 0          <-- and wakes the joiner
189 exit_group(0)                          = ?
```

A **detached** thread (`pthread_detach` or `PTHREAD_CREATE_DETACHED`) cannot be joined; its resources are released automatically when it exits. A thread that finishes but is never joined or detached leaks its stack and descriptor until the process ends (a "zombie thread"). To make creating threads fast, NPTL **caches** stacks of finished threads (up to a size limit) and reuses them instead of calling `mmap/munmap` again.

WATCH OUT

Passing the address of a local variable to a thread is a classic bug. In Lab 1 we pass `&ids[i]`, which lives in `main`'s stack and remains valid because `main` joins the threads. If `main` returned first, the threads would read freed stack memory. And never pass `&i` of a loop counter: by the time the thread reads it, `i` has already changed.

CHAPTER 7

Futexes: Fast Locks With Kernel Help

7.1 The problem

A mutex must do two things: (1) let one thread in at a time; (2) make waiting threads sleep rather than burn CPU. Consider two naive designs:

- **Always ask the kernel** (e.g., a system call for every lock/unlock). Correct, but each system call costs hundreds of nanoseconds or more, even when nobody else wants the lock. In real programs, most lock operations are *uncontended*.
- **Pure spinning** in user space. Fast when uncontended, but if the lock holder is descheduled, every waiter wastes its whole time slice spinning.

What we want: **stay in user space when there is no contention, and only enter the kernel when a thread truly has to sleep or wake someone**. This is exactly what a **futex** ("fast userspace mutex") provides.

7.2 What a futex is

A futex is not a lock object. It is just **an integer in ordinary user memory** (the address is the identity) plus two kernel operations on that address:

- `futex(addr, FUTEX_WAIT, val):` //If `*addr` still equals `val`, put me to sleep on `addr`. Otherwise return immediately.// The check-and-sleep is **atomic** in the kernel, so a wake-up cannot slip in between and be lost.
- `futex(addr, FUTEX_WAKE, n):` //Wake up at most `n` threads sleeping on `addr`.//

The kernel only creates queue state when a thread actually calls `FUTEX_WAIT`. It keeps waiters in a hash table keyed by the address (for process-private futexes, by address within that process). Locking logic (the integer's meaning) is entirely up to the user-space library.

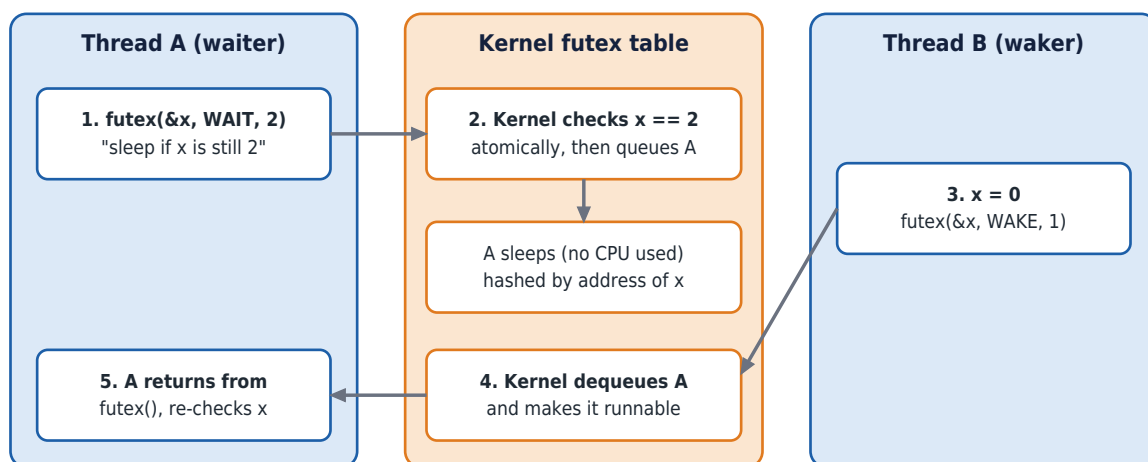


Figure 7.1: A futex wait and wake. The kernel's only job is the atomic "check and sleep" and the "wake".

7.3 A mutex built on a futex

The classic three-state design (from Ulrich Drepper's paper *Futexes Are Tricky*) stores one integer in the mutex: **0** = unlocked, **1** = locked and nobody waiting, **2** = locked and there may be waiters. State 2 exists so that **unlock** knows whether it needs the expensive **futex(WAKE)** at all.

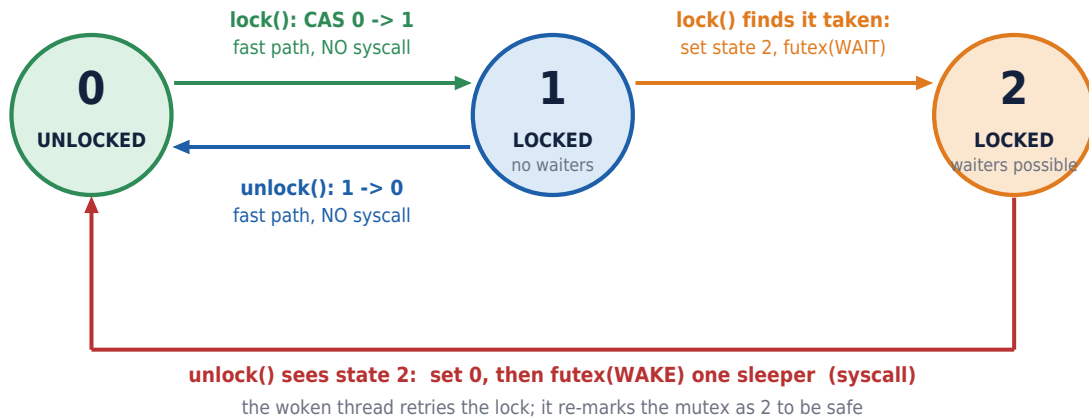


Figure 7.2: Three-state futex mutex. Only the red and orange transitions involve the kernel.

```
// state: 0 = unlocked, 1 = locked (no waiters), 2 = locked (waiters possible)
// cmpxchg(p, old, new) atomically sets *p = new if *p == old; returns the previous value.

void lock(int *m) {
    int c = cmpxchg(m, 0, 1);           // FAST PATH: try 0 -> 1. No syscall if it works.
    if (c != 0) {                       // somebody holds it: slow path
        do {
            // make sure the state says "waiters exist" (2), then sleep while it stays 2
            if (c == 2 || cmpxchg(m, 1, 2) != 0)
                futex_wait(m, 2);       // syscall: sleep if *m == 2
        } while ((c = cmpxchg(m, 0, 2)) != 0); // woke up: try to take it (as state 2)
    }
}

void unlock(int *m) {
    if (atomic_dec(m) != 1) {           // old value was 2: someone may be sleeping
        *m = 0;
        futex_wake(m, 1);              // syscall: wake one sleeper
    }                                  // old value was 1: nobody waiting, done, no syscall
}
```

The real glibc code ([nptl/pthread_mutex_lock.c](#), [sysdeps/nptl/lowlevellock.h](#)) is more elaborate. It supports recursive, error-checking, adaptive-spinning, robust, priority-inheritance and process-shared mutexes, but the heart is this same idea.

Lab 3 lets you verify it. The uncontended lock and unlock produce **no** **futex** calls. The contended one produces exactly one wait and one wake:

```
189 futex(0x55859ffcb040, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
190 futex(0x55859ffcb040, FUTEX_WAKE_PRIVATE, 1) = 1
189 <... futex resumed> = 0
```

Line 1: the main thread (TID 189) sleeps, and the 2 is the mutex state it saw. Line 2: the holder thread (TID 190) unlocks and wakes one sleeper. Line 3: main wakes up and retries the lock.

`_PRIVATE` means the futex is only used within one process, which allows the kernel to use a faster lookup (address in this process's memory) than for futexes shared across processes through shared memory.

7.4 Everything else builds on futexes

- **Condition variables** (`pthread_cond_wait/signal/broadcast`): sequence counters plus futex wait/wake.
- **Semaphores** (`sem_wait/sem_post`): a counter with futex sleep when it reaches zero.
- **Read-write locks, barriers, `pthread_once`, and `pthread_join`** all use futexes as their sleeping mechanism.
- Mutex attributes select behaviour: **normal**, **recursive** (same thread may relock), **error-checking** (detect misuse), **adaptive** (spin briefly before sleeping), **robust** (recover if the owner dies), **priority-inheritance** (avoid priority inversion), and **process-shared** (usable across processes via shared memory).

KEY IDEA

Cost model: uncontended lock/unlock = a couple of atomic instructions (nanoseconds, no kernel).
Contended = a system call, a sleep, and a context switch. So the way to make multithreaded programs fast is to *reduce contention*: hold locks briefly, and split one big lock into smaller ones.

CHAPTER 8

Signals, Cancellation, `fork()`, and `exec()`

Operating-system features were designed for single-threaded processes. POSIX had to say what each means when there are several threads. This is where the thread-group support in the kernel matters most.

8.1 Signals

- **Handlers are per process** (shared `sigand_struct`). One `sigaction()` call affects all threads.
- **The signal mask is per thread**. Each thread chooses which signals it blocks (`pthread_sigmask`).
- **Process-directed signals** (sent by `kill()`, or generated like `SIGINT/SIGTERM`) are delivered to *any one* thread that does not block them. The kernel keeps them in a shared pending set.
- **Thread-directed signals** (`pthread_kill()`, implemented with `tgkill`) go to one chosen thread. Synchronous signals like `SIGSEGV` or `SIGFPE` go to the thread that caused them.
- **Default actions affect the entire process**. An unhandled `SIGSEGV` kills all threads. `SIGSTOP` stops them all.

TRY IT

A common, robust design: **block all signals in every thread** (before creating threads), then dedicate one thread to loop on `sigwait()` and handle signals as ordinary code. That avoids the hard restrictions on what can be done inside a signal handler.

8.2 Thread cancellation

`pthread_cancel(t)` asks thread `t` to terminate. NPTL implements it by sending `t` an internal real-time signal (`SIGCANCEL`) with `tgkill`. The library's handler for that signal decides what to do. By default, cancellation is **deferred**: it only takes effect when the thread reaches a **cancellation point**, a function like `read`, `write`, `sleep`, `pthread_cond_wait` or `pthread_testcancel`. Cleanup handlers registered with `pthread_cleanup_push` then run.

WATCH OUT

Cancellation can leave locks held or data half-updated. Most modern code avoids `pthread_cancel` and instead sets a shared "please stop" flag (or writes to a pipe or eventfd) that the worker checks regularly.

8.3 Internal signals used by NPTL

NPTL reserves two real-time signals for its own use: `SIGCANCEL` (for cancellation) and `SIGSETXID`. That is why user programs get `SIGRTMIN` = 34 on glibc rather than 32. `SIGSETXID` solves an interesting mismatch: POSIX says `setuid()` changes the user ID of the whole *process*, but the Linux system call only changes the calling *thread*. So glibc signals every thread and each one performs the system call itself, keeping all threads' credentials consistent.

8.4 `fork()` in a multithreaded program

- `fork()` creates a child containing **only the thread that called `fork`**. The other threads simply do not exist in the child.
- If another thread held a mutex at the moment of `fork` (for example, the internal `malloc` lock), that mutex stays locked in the child forever, and nothing can ever unlock it: **deadlock**.
- POSIX therefore says that in a multithreaded process, the child may only call *async-signal-safe* functions until it calls `exec`. `pthread_atfork()` lets a library register handlers to make locks safe around `fork` (glibc does this for its own internals).

8.5 `exec()` and `exit`

- `exec*()` replaces the whole program. The kernel first **destroys every other thread** in the group (in `de_thread()`, `fs/exec.c`), and the calling thread takes over the leader's PID.
- Returning from `main()` or calling `exit()` calls `exit_group()`: all threads end.
- `pthread_exit()` in the *main* thread ends only that thread; the process keeps running until the last thread finishes.

CHAPTER 9

Thread-Local Storage (TLS)

Some data must be per-thread even though the address space is shared. The best-known example is `errno`. If thread A's `read()` fails while thread B's `open()` succeeds, they must not overwrite each other's error code. In multithreaded programs `errno` is not a plain global; it is a macro that expands to a per-thread location.

9.1 Using TLS

```
__thread int request_count = 0;    // GCC/Clang extension (C11: _Thread_local; C23: thread_local)

void *worker(void *arg) {
    request_count++;               // each thread increments ITS OWN copy
    return NULL;
}
```

Each thread starts with its own copy of `request_count` (initialized to 0). For dynamic, run-time created per-thread values, POSIX has `pthread_key_create()` / `pthread_setspecific()` / `pthread_getspecific()`, with optional destructor functions called when the thread exits.

9.2 How the hardware and kernel help

Reading a `__thread` variable must be as cheap as reading an ordinary one, so it cannot involve a table lookup by thread ID. The trick uses a **dedicated CPU register that always points to the current thread's data**:

- On **x86-64**, the **FS** segment base register points to the current thread's TCB (`struct pthread`). A TLS variable is accessed as something like `mov %fs:offset, %eax`: one instruction.
- On **AArch64** the same role is played by the **TPIDR_EL0** register; on 32-bit x86, by **GS**.
- **The kernel's role**: `CLONE_SETTLS` tells `clone()` what value to load into that register for the new task, and the kernel's context-switch code **saves and restores** the register for each task, so the value is always right for whichever thread is running on a CPU.

The TCB's first field is a pointer to itself, so `pthread_self()` can be answered by reading it directly. That is why the value printed by `pthread_self()` in Lab 1 looks like a big memory address (for example `139798709487424`): it is an address, the thread's descriptor, and is unrelated to the kernel TID.

KEY IDEA

TLS is a team effort across three layers: the **compiler** emits `%fs:offset` accesses, the **library** lays out and initializes each thread's TLS block next to its TCB, and the **kernel** loads and preserves the thread-pointer register per task.

CHAPTER 10

Scheduling and System Limits

10.1 The kernel schedules threads

Because NPTL is 1:1, **there is no library scheduler**. Each thread is a task, and the kernel scheduler treats it like any other. In current kernels, normal threads use the fair scheduler (CFS from 2.6.23, replaced by EEVDF in 6.6): each runnable task gets a share of CPU time, balanced across cores. Threads of one program can run on different cores at the same instant.

Why switching between threads of one process is cheaper

A context switch between tasks of *different* processes must also switch the address space (load a different page-table root, and lose cached address translations in the TLB). Between two threads of the *same* process, the address space is identical, so the kernel can skip that part. This is one of the reasons multithreading is often more efficient than multiprocessing.

10.2 Controls you can use

Feature	pthread call	Underlying kernel mechanism
Scheduling policy and priority	<code>pthread_setschedparam</code> , attribute <code>pthread_attr_setschedpolicy</code>	<code>sched_setscheduler</code> . Policies: <code>SCHED_OTHER</code> (normal), <code>SCHED_FIFO/SCHED_RR</code> (real-time), <code>SCHED_BATCH</code> , <code>SCHED_IDLE</code> , <code>SCHED_DEADLINE</code>
CPU affinity	<code>pthread_setaffinity_np</code> , <code>pthread_attr_setaffinity_np</code>	<code>sched_setaffinity</code> on the thread's TID
Contention scope	<code>pthread_attr_setscope</code>	Only <code>PTHREAD_SCOPE_SYSTEM</code> is supported (the kernel schedules against all threads). Asking for <code>SCOPE_PROCESS</code> fails.
Thread name (debugging)	<code>pthread_setname_np</code>	<code>prctl(PR_SET_NAME)</code> ; shows in <code>top -H</code>
Stack size	<code>pthread_attr_setstacksize</code>	Library chooses the <code>mmap</code> size (Chapter 6)

NOTE

On Linux, **nice** values are per **thread**, not per process. The POSIX standard says per process, so this is a known non-conformance documented in the `nptl(7)` man page.

10.3 Why does `pthread_create` fail? (EAGAIN)

Creating thousands of threads eventually hits a ceiling. When `pthread_create` returns `EAGAIN` ("Resource temporarily unavailable") one of these limits is the likely cause:

Limit	Where to look	Explanation
Per-user process/thread count	<code>ulimit -u (RLIMIT_NPROC)</code>	Every thread counts as a task toward this limit.

Limit	Where to look	Explanation
System-wide thread count	<code>/proc/sys/kernel/threads-max</code>	Kernel-wide cap on tasks.
PID/TID space	<code>/proc/sys/kernel/pid_max</code>	Each thread consumes an ID; all IDs share the same number space.
Memory mappings	<code>/proc/sys/vm/max_map_count</code>	Each thread stack needs at least two mappings (stack + guard page).
Virtual address space / RAM	<code>ulimit -v, free -h</code>	8 MB stack x 10,000 threads = 80 GB of <i>virtual</i> space, plus kernel memory per task (kernel stack, <code>task_struct</code>).
Container / cgroup limit	<code>pids.max</code> in the cgroup	Docker/Kubernetes can cap the number of tasks per container.

TRY IT

If you need very many concurrent operations (tens of thousands of connections), consider an event loop (`epoll`, `io_uring`) or a smaller pool of worker threads rather than one thread per task. Thread pools also avoid paying creation cost repeatedly.

CHAPTER 11

Hands-On Labs

Try these on any Linux machine (a VM or WSL2 works). Compile every program with `gcc -pthread file.c -o name`. The outputs shown were captured on a real system.

Lab 1: Same PID, different TIDs

Goal: confirm that threads share a PID (kernel TGID) but have unique TIDs, and that `pthread_self()` is something else entirely.

```
#define _GNU_SOURCE
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

void *worker(void *arg) {
    int id = *(int *)arg;
    printf("worker %d: getpid()=%d  gettid()=%ld  pthread_self()=%lu\n",
        id, getpid(), (long)syscall(SYS_gettid), (unsigned long)pthread_self());
    return NULL;
}

int main(void) {
    pthread_t t[2];
    int ids[2] = {1, 2};
    printf("main : getpid()=%d  gettid()=%ld  pthread_self()=%lu\n",
        getpid(), (long)syscall(SYS_gettid), (unsigned long)pthread_self());
    for (int i = 0; i < 2; i++) pthread_create(&t[i], NULL, worker, &ids[i]);
    for (int i = 0; i < 2; i++) pthread_join(t[i], NULL);
    return 0;
}
```

```
main : getpid()=125  gettid()=125  pthread_self()=139798709487424
worker 1: getpid()=125  gettid()=126  pthread_self()=139798706648768
worker 2: getpid()=125  gettid()=127  pthread_self()=139798698256064
```

Observe: the main thread's TID equals the PID (it is the group leader). Workers have new TIDs 126 and 127, but `getpid()` still says 125. Recall Section 4.3.

CHECK YOURSELF

Why are the `pthread_self()` values so large, and why do they differ from the TIDs? (Hint: Section 9.2.)

Lab 2: Make a race condition, then fix it with a mutex

Goal: see lost updates, then see how a mutex (futex underneath) prevents them.

```
#include <pthread.h>
#include <stdio.h>

#define NTHREADS 4
#define ITERS 20000000

long counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

```

void *unsafe_worker(void *arg) {
    for (int i = 0; i < ITTERS; i++) counter++;          /* data race! */
    return NULL;
}
void *safe_worker(void *arg) {
    for (int i = 0; i < ITTERS; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
static void run(void *(*fn)(void *), const char *label) {
    pthread_t t[NTHREADS];
    counter = 0;
    for (int i = 0; i < NTHREADS; i++) pthread_create(&t[i], NULL, fn, NULL);
    for (int i = 0; i < NTHREADS; i++) pthread_join(t[i], NULL);
    printf("%-8s expected=%d actual=%ld\n", label, NTHREADS * ITTERS, counter);
}
int main(void) {
    run(unsafe_worker, "unsafe");
    run(safe_worker, "safe");
    return 0;
}

```

```

unsafe   expected=800000000  actual=29981162
safe     expected=800000000  actual=800000000

```

Observe: the unsafe version loses over half the increments; the mutex version is exactly right. Compile without optimization (`-O0`) to make the race easier to see. The exact wrong value is different on every run. With a small `ITTERS` (or if the compiler is allowed to optimize the loop into a register), you may get lucky and see the right answer. **Correct by luck is still a bug.** Try timing the two versions with `time ./lab2`. Why is the safe one slower?

Lab 3: Watch clone and futex with strace

Goal: see the system calls behind `pthread_create`, `pthread_join` and a contended mutex. Install `strace` if needed (`sudo apt install strace`).

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void *holder(void *arg) {
    pthread_mutex_lock(&m);
    usleep(200000);          /* hold the lock for 0.2 s */
    pthread_mutex_unlock(&m);
    return NULL;
}

int main(void) {
    pthread_t t;
    pthread_mutex_lock(&m);    /* uncontended: expect NO futex call */
    pthread_mutex_unlock(&m);

    pthread_create(&t, NULL, holder, NULL);
    usleep(50000);           /* let holder grab the lock */
    pthread_mutex_lock(&m);    /* contended: expect futex(WAIT) */
    pthread_mutex_unlock(&m);
    pthread_join(t, NULL);
    return 0;
}

$ strace -f -e trace=clone3,futex,mmap,mprotect,exit,exit_group ./lab3
```

What to look for, matching the output shown in Chapters 6 and 7:

- 1 An `mmap(... PROT_NONE ... MAP_STACK)` of 8392704 bytes, followed by `mprotect` making all but the lowest page read-write (that is the guard page).
- 2 A `clone3` with `CLONE_VM|...|CLONE_THREAD|...`
- 3 No `futex` line for the first, uncontended lock/unlock.
- 4 A `FUTEX_WAIT_PRIVATE ... 2` in main and a matching `FUTEX_WAKE_PRIVATE` from the holder thread.
- 5 A `FUTEX_WAIT_BITSET` on the TID word during `pthread_join`, woken when the thread's `exit` happens.

Lab 4: Look at threads from outside

Goal: find threads in `ps` and `/proc`.

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void *sleeper(void *arg) { sleep(30); return NULL; }

int main(void) {
    pthread_t t[3];
    printf("pid = %d\n", getpid());
    fflush(stdout);
    for (int i = 0; i < 3; i++) pthread_create(&t[i], NULL, sleeper, NULL);
    for (int i = 0; i < 3; i++) pthread_join(t[i], NULL);
    return 0;
}
```

```

$ ./lab4 &                                     # note the printed pid, e.g. 223
$ ps -T -p 223
  PID  SPID  TTY          TIME CMD
  223   223  ?            00:00:00 lab4
  223   225  ?            00:00:00 lab4
  223   226  ?            00:00:00 lab4
  223   227  ?            00:00:00 lab4

$ ls /proc/223/task
223 225 226 227

$ grep -E "(Name|Tgid|Pid|Threads)" /proc/223/status
Name:   lab4
Tgid:   223
Pid:    223
Threads:      4

$ for t in $(ls /proc/223/task); do echo -n "task $t: "; \
  grep -E "(Pid|Tgid)" /proc/223/task/$t/status | tr '\n' ' '; echo; done
task 223: Tgid: 223 Pid: 223
task 225: Tgid: 223 Pid: 225
task 226: Tgid: 223 Pid: 226
task 227: Tgid: 223 Pid: 227

```

Observe: four tasks (main plus three sleepers) share TGID 223. The kernel's field called **Pid** is the *thread* ID, matching the naming table in Section 4.3. Try `top -H -p 223` and `pkill -9 lab4` (one signal kills the whole group). If you want a challenge, run `cat /proc/223/maps` and find the three 8 MB stack regions and their guard pages.

CHAPTER 12

Modern Developments and Alternatives

12.1 What changed since 2003?

- **libpthread merged into libc (glibc 2.34, 2021).** Older programs linked to `libpthread.so`; today the functions live in `libc.so`, and `-pthread` is mostly kept for compatibility. NPTL is still the implementation, just packaged differently.
- `clone3()` (kernel 5.3) gives a cleaner, extensible interface; glibc uses it when available.
- **Scheduler evolution:** O(1), then CFS, then EEVDF, all supporting the 1:1 model unchanged.
- **More futex features:** priority inheritance, robust lists, requeue operations that avoid "thundering herd" wake-ups, and `futex_waitv()` for waiting on several futexes at once (used by compatibility layers for games).
- **cgroups and containers:** thread counts can now be limited per container (`pids.max`).
- **C11 and C++11** added standard threading (`<threads.h>`, `std::thread`). On Linux with glibc, these are thin wrappers around the same NPTL/pthread machinery.

12.2 Other C libraries

NPTL is specific to **glibc**. Other libraries have their own pthreads implementations. **musl** libc (used by Alpine Linux) has its own, much smaller design; it also uses `clone` and futexes, and is another 1:1 implementation, but the code is entirely different. **Android's Bionic** is another separate implementation. All of them present the same POSIX API to applications.

12.3 Alternatives to "one thread per task"

Approach	Idea	Trade-off
Thread pool	Reuse a fixed number of threads for many tasks	Avoids creation cost and thread explosion; you manage the queue
Event loop (<code>epoll</code> , <code>io_uring</code>)	One or few threads handle many connections through readiness/completion events	Scales to very large connection counts; code is harder to structure
Green threads / coroutines / goroutines	Language runtime schedules lightweight tasks on a few kernel threads (M:N)	Very cheap concurrency; needs runtime support; blocking system calls need care
Multiple processes	Use <code>fork()</code> or separate programs; communicate with pipes/sockets/shared memory	Strong isolation and fault containment; communication more expensive

CHAPTER 13

Review, Misconceptions, and Reference

13.1 Common misconceptions

Misconception	Reality
"NPTL is a kernel feature."	NPTL is a user-space library (part of glibc). It depends on kernel features such as <code>clone</code> flags and <code>futexes</code> .
"Linux threads are different kernel objects from processes."	Both are <code>task_struct</code> . The difference is what they share.
" <code>getpid()</code> returns a unique number per thread."	It returns the TGID, the same for all threads. <code>gettid()</code> is unique per thread.
" <code>pthread_self()</code> is the kernel thread ID."	It is a library handle (the descriptor's address on glibc).
"Taking a mutex always requires a system call."	Uncontended lock/unlock is entirely user-space atomic operations.
"More threads always means faster."	Beyond the number of cores (for CPU-bound work), extra threads add switching and contention overhead.
"Each thread stack uses 8 MB of RAM."	8 MB is reserved virtual space; only touched pages use RAM.
" <code>fork()</code> copies all threads."	Only the calling thread exists in the child.
"Threads are always better than processes."	Threads lose isolation: a bug in one thread can crash or corrupt the whole process.

13.2 Where to find it in the source

Paths may shift between versions; use them as starting points for reading.

Topic	Linux kernel	glibc
Thread creation	<code>kernel/fork.c: kernel_clone(), copy_process()</code>	<code>nptl/pthread_create.c</code> , <code>nptl/allocatestack.c</code> , <code>sysdeps/unix/sysv/linux/clone3.S</code>
Thread descriptor / TLS	<code>arch/x86/kernel/process_64.c</code> (FS base handling)	<code>nptl/descr.h</code> (<code>struct pthread</code>)
Task structure	<code>include/linux/sched.h: struct task_struct</code>	n/a
Exit	<code>kernel/exit.c: do_exit(), do_group_exit()</code>	<code>nptl/pthread_exit.c</code> , <code>nptl/nptl_deallocate_tsd.c</code>
Futex	<code>kernel/futex/</code> (older kernels: <code>kernel/futex.c</code>)	<code>sysdeps/nptl/lowlevellock.h</code> , <code>nptl/pthread_mutex_lock.c</code>
Signals	<code>kernel/signal.c: tgkill</code> , delivery to thread groups	<code>nptl/pthread_kill.c</code> , <code>nptl/pthread_cancel.c</code>
exec vs threads	<code>fs/exec.c: de_thread()</code>	n/a

13.3 Review questions

- 1 State the difference between POSIX Threads and NPTL.
- 2 Name the three threading models and explain why NPTL uses 1:1. What is the main weakness of M:1?
- 3 In Linux, what is the only real difference between a process and a thread?
- 4 List five `clone()` flags used to create a thread and describe what each does.
- 5 Explain the relationship between PID, TGID and TID on Linux. What do `getpid()` and `gettid()` return in a non-leader thread?
- 6 Why did LinuxThreads need a manager thread and why does NPTL not?
- 7 Describe what `pthread_create` does in user space, then in the kernel.
- 8 What is a guard page and what problem does it solve?
- 9 Explain how `pthread_join` blocks and how it is woken up, without any signals.
- 10 What does `futex(addr, FUTEX_WAIT, val)` do? Why must the check of `*addr` and the sleep be atomic?
- 11 In the three-state mutex, why is state 2 needed? Under what condition does `unlock` avoid a system call?
- 12 A multithreaded program calls `fork()`. What happens to the other threads, and what danger does this create?
- 13 How does the kernel help make `errno` thread-safe?
- 14 Why can switching between two threads of the same process be cheaper than switching between two processes?
- 15 `pthread_create` returns `EAGAIN` after about 32,000 threads. Name three limits that could be responsible.

13.4 Answer key

- 1 **POSIX Threads** is the API specification; **NPTL** is the Linux/glibc implementation of it.
- 2 M:1, 1:1, M:N. NPTL uses 1:1 because the Linux kernel already creates and schedules tasks cheaply, and 1:1 gives true parallelism without a complex two-level scheduler. M:1 uses only one CPU, and one blocking system call blocks all threads.
- 3 What they share. Threads are tasks that share address space, file table, signal handlers, and thread group. Both are `task_struct`.
- 4 `CLONE_VM` (share memory), `CLONE_FILES` (share file descriptors), `CLONE_FS` (share cwd/umask), `CLONE_SIGHAND` (share signal handlers), `CLONE_THREAD` (same thread group), plus `CLONE_SETTLS`, `CLONE_SYSVSEM`, `CLONE_PARENT_SETTID`, `CLONE_CHILD_CLEARTID`.
- 5 PID = TGID: the same for all threads of a process (the TID of the first thread). TID: unique per task. In a non-leader thread `getpid()` returns the group's TGID (equal to the leader's TID) and `gettid()` returns that thread's own, different, TID.
- 6 The old kernel had no thread groups or proper signal/wait semantics, so the library emulated them with a manager thread and internal signals. NPTL relies on the kernel's thread groups, `exit_group`, `tgkill` and `futexes` instead.
- 7 User space: get/allocate stack (`mmap`), guard page (`mprotect`), initialize `struct pthread` and TLS, call `clone3`. Kernel: `copy_process` creates a new task sharing mm/files/sighand with the same TGID and a new TID, sets TLS, and makes it runnable. New thread starts in `start_thread`.
- 8 An inaccessible page at the low end of the stack. Overflowing the stack faults immediately (`SIGSEGV`) rather than silently corrupting adjacent memory.
- 9 `pthread_join` calls `futex(WAIT)` on the target's TID word. `CLONE_CHILD_CLEARTID` makes the kernel zero that word and `futex-wake` on the target's exit.

- 10 If `*addr == val`, sleep on `addr`; otherwise return. It must be atomic so that a wake-up happening between the check and the sleep cannot be lost, which would leave the sleeper blocked forever.
- 11 State 2 records that waiters may exist. `unlock` avoids the system call when the old state was 1 (locked, no waiters).
- 12 Only the calling thread exists in the child. A mutex held by another thread at `fork` time stays locked forever in the child, which can cause deadlock (e.g. in `malloc`). Only async-signal-safe calls are permitted before `exec`.
- 13 Each thread has its own thread-pointer register (FS base on x86-64) pointing at its TCB. The kernel loads it at `clone` (`CLONE_SETTLS`) and saves/restores it on every context switch; `errno` is located relative to it.
- 14 Threads of one process share the address space, so the kernel need not switch page tables or lose cached address translations.
- 15 Any of: `ulimit -u (RLIMIT_NPROC)`, `kernel.threads-max`, `kernel.pid_max` (default 32768 on many systems), `vm.max_map_count`, available memory or address space, or a cgroup `pids.max`.

13.5 Glossary

Term	Meaning
Address space	The range of virtual memory addresses a process can use.
Atomic operation	A CPU instruction sequence that completes as one indivisible step (e.g. compare-and-swap).
Context switch	Saving one task's CPU state and loading another's so the CPU can run a different task.
Contention	Several threads trying to take the same lock at the same time.
Futex	"Fast userspace mutex": a kernel facility to sleep/wake threads on a user-space integer.
glibc	The GNU C Library, the standard C library on most Linux distributions. Contains NPTL.
Guard page	An inaccessible page at the end of a stack to catch overflows.
Race condition	A bug where the result depends on unpredictable timing of threads.
Task	The kernel's schedulable unit, <code>struct task_struct</code> ; used for both processes and threads.
TCB	Thread control block: glibc's per-thread descriptor (<code>struct pthread</code>).
TGID	Thread group ID: the value <code>getpid()</code> returns.
TID	Thread ID: the unique task number (<code>gettid()</code>).
TLS	Thread-local storage: variables with a separate instance per thread.
Thread group	The set of tasks created with <code>CLONE_THREAD</code> that form one process.
1:1 model	Each user-level thread corresponds to exactly one kernel task.

13.6 References and further reading

- **Linux manual pages:** `pthreads(7)`, `nptl(7)`, `clone(2)`, `futex(2)`, `futex(7)`, `gettid(2)`, `signal(7)`, `pthread_create(3)`, `pthread_mutex_lock(3)`. Read them with `man 7 nptl`.
- Ulrich Drepper and Ingo Molnar. *The Native POSIX Thread Library for Linux*. Red Hat, 2003. The original design document.
- Ulrich Drepper. *Futexes Are Tricky*. The paper behind the mutex algorithm in Chapter 7.

- Hubertus Franke, Rusty Russell, Matthew Kirkwood. *Fuss, Futexes and Furwocks: Fast Userlevel Locking in Linux*. Ottawa Linux Symposium, 2002.
- Silberschatz, Galvin, Gagne. *Operating System Concepts*. The chapters on threads and synchronization (threading models, Linux `clone`).
- Michael Kerrisk. *The Linux Programming Interface*. The chapters on POSIX threads.
- Robert Love. *Linux Kernel Development*. Process management and scheduling chapters.
- Kernel source (kernel.org) and glibc source (sourceware.org/glibc): files listed in Section 13.2.

NOTE — LAST WORD

This guide was prepared as a study aid. Kernel and glibc details evolve; when in doubt, consult the manual pages and source for the version you are running. **Generated for CMSC 125 ICS UPLB · Reviewed by JACH**